# ATS Resume Tailoring Tool: Selection Algorithm & Data Pipeline

Po Haoting

March 2026

## Contents

**1 Overview** **2**
1.1 Index Dimensions . . . 2
1.2 Pipeline Architecture . . . 2

**2 Data Processing Pipeline** **2**
2.1 Stage 1: Chunking & Loading . . . 2
2.2 Stage 2: Embedding . . . 3
2.3 Stage 3: Hybrid Scoring . . . 3
2.3.1 Semantic Score: Max-Pooled Cosine Similarity . . . 3
2.3.2 Keyword Bonus: Exact Match in JD Text . . . 3
2.3.3 Recency Multiplier . . . 4

**3 Selection Algorithm** **4**
3.1 Experience Selection . . . 4
3.2 Project Selection . . . 4
3.3 Skill Selection . . . 4
3.4 Certification Selection . . . 5
3.5 Page Budget Enforcement . . . 5

**4 Output Generation** **5**
4.1 LaTeX Resume . . . 5
4.2 LLM Rewriting Prompt . . . 5
4.3 Match Visualization . . . 6

**5 Design Evolution** **6**
5.1 Iteration 1: Pure Cosine Similarity . . . 6
5.2 Iteration 2: Diversity Filter Removal . . . 6
5.3 Iteration 3: Hybrid Scoring . . . 6
5.4 Iteration 4: Validated Results . . . 7

**6 Limitations & Future Work** **7**

# 1 Overview

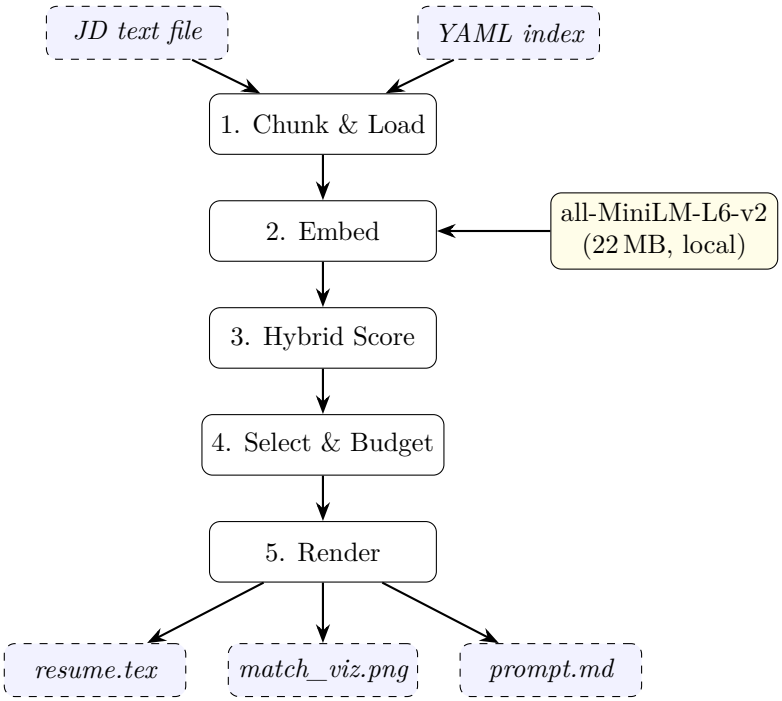
The tailoring tool generates a 1-page A4 LaTeX resume from a structured YAML index, ranked against a job description (JD) using semantic embeddings and keyword matching. The entire pipeline runs locally with no API calls.

## 1.1 Index Dimensions

| Category | Count | Source File |
|---|---|---|
| Skills (across 12 categories) | 56 | `index/skills.yaml` |
| Projects | 19 | `index/projects.yaml` |
| Experience roles | 5 | `index/experience.yaml` |
| Certifications | 5 | `index/certifications.yaml` |

Table 1: YAML index inventory.

## 1.2 Pipeline Architecture




# 2 Data Processing Pipeline

## 2.1 Stage 1: Chunking & Loading

The JD text is split into sentence-level chunks for embedding. Each line is split on "`. `" (period followed by space) to produce sentence fragments. Empty lines are discarded. This produces a set of $n$ JD chunks $\{c_1, c_2, \ldots, c_n\}$.

All four YAML files are parsed into Python dictionaries. Education is truncated to the two most recent entries.

## 2.2 Stage 2: Embedding

Each JD chunk and each index item is encoded into a 384-dimensional vector using `all-MiniLM-L6-v2` (a sentence-transformers model, 22 MB, runs on CPU in <2 s for the full index).

**Text representations** for embedding are constructed as follows:

| Item Type | Text Construction |
|---|---|
| Skill | `"{name} {ats_keywords joined}"` |
| Project | `"{name} {summary} {ats_tags} {tech_stack all fields} {responsibilities}"` |
| Role | `"{title} {company} {skills_used} {ats_tags} {bullets}"` |
| Certification | `"{name} {issuer} {ats_keywords joined}"` |

Table 2: Text representation per item type. Fields are space-concatenated.

## 2.3 Stage 3: Hybrid Scoring

Each item receives a **hybrid score** combining three signals:

$$\text{score}(i) = \big(\underbrace{S_{\text{sem}}(i)}_{\text{semantic}} + \underbrace{S_{\text{kw}}(i)}_{\text{keyword}}\big) \times \underbrace{R(i)}_{\text{recency}}$$

### 2.3.1 Semantic Score: Max-Pooled Cosine Similarity

Given JD chunk embeddings $\{\mathbf{c}_1, \ldots, \mathbf{c}_n\}$ and item embedding $\mathbf{v}_i$:

$$S_{\text{sem}}(i) = \max_{j \in \{1,\ldots,n\}} \frac{\mathbf{c}_j \cdot \mathbf{v}_i}{\|\mathbf{c}_j\| \, \|\mathbf{v}_i\|}$$

Max-pooling (rather than mean-pooling) ensures that an item is rewarded for matching *any* part of the JD strongly, even if most JD sentences are unrelated.

### 2.3.2 Keyword Bonus: Exact Match in JD Text

For each item's ATS keyword list $K_i = \{k_1, k_2, \ldots\}$, count how many appear literally in the lowercased JD text:

$$m_i = \big|\{k \in K_i : k \text{ appears in JD (case-insensitive, word-boundary)}\}\big|$$

$$S_{\text{kw}}(i) = \min(0.06 \times m_i,\ 0.20)$$

The cap at 0.20 prevents keyword-heavy items from dominating purely on keyword count. Word-boundary matching is used for alphanumeric keywords (so "C" does not match "Computer"), while special-character keywords ("C++", "C#", ".NET") use substring matching.

**Rationale.** Semantic similarity alone underscores generic but critical terms. The embedding for "Python" is diffuse—it encodes the concept broadly rather than matching the literal string. A JD that explicitly says "Python" should boost the Python skill more than one that merely discusses data pipelines.

#### 2.3.3 Recency Multiplier

The end date is parsed from the period string (e.g., "Oct 2025 – Present" → current, "2021 – 2022" → 2022):

$$R(i) = \begin{cases} 1.10 & \text{if end date is current or future ("Present")} \\ 1.05 & \text{if ended within past 12 months} \\ 1.02 & \text{if ended within past 24 months} \\ 1.00 & \text{otherwise} \end{cases}$$

**Rationale.** Recent projects and roles are more relevant to hiring managers. A 10% boost for ongoing work and 5% for recent work is enough to break ties between similarly-scored items without overriding a strong semantic match from an older project.

Certifications receive keyword bonuses but no recency multiplier (they do not expire in relevance).

# 3 Selection Algorithm

## 3.1 Experience Selection

**Algorithm 1** Select Experience Roles

**Require:** Roles $\mathcal{R}$, hybrid scores, $k = 2$

```
Sort R by score descending
return top k roles
```

## 3.2 Project Selection

**Algorithm 2** Select Projects

**Require:** Projects $\mathcal{P}$, hybrid scores, $k = 4$

```
Sort P by score descending
return top k projects
```

Selection is purely score-based. An earlier version used a tech-stack diversity filter (Section 5), which was removed after empirical testing showed it caused more harm than good.

## 3.3 Skill Selection

Skill selection uses a two-level process:

**Algorithm 3** Select Skills

**Require:** Skill categories $\mathcal{C}$, JD embeddings, JD text, $L = 4$ lines

```
for each category c ∈ C do
    for each skill s ∈ c do
        Compute hybrid score: score(s) = S_sem(s) + S_kw(s)
    end for
    Sort skills by (proficiency rank, −score)        ▷ advanced → intermediate → familiar
    Compute category relevance: s̄_c = mean of top-5 scores
end for
Line 1 (always): Programming Languages, ordered by proficiency then score
Lines 2–L: Top categories by s̄_c, each showing skills above threshold (> 0.15)
return L formatted skill lines
```

**Proficiency ordering.** Within each skill line, skills are sorted by proficiency level first (advanced before intermediate before familiar), then by hybrid score as tiebreaker. This ensures that the candidate's strongest skills appear first on the resume, while still being filtered by JD relevance.

| Proficiency | Rank | Meaning |
|---|---|---|
| Advanced | 0 | Production systems, can mentor |
| Intermediate | 1 | Project use, independent |
| Familiar | 2 | Coursework or exposure |

Table 3: Proficiency ranking used for skill ordering.

### 3.4 Certification Selection

Certifications are selected by a hard threshold on the hybrid score:

$$\text{selected} = \{c : \text{score}(c) > 0.25\}$$

This threshold was chosen empirically—certifications scoring below 0.25 tend to be irrelevant to the target role.

### 3.5 Page Budget Enforcement

After selection, a line-count estimate determines whether the content fits on one A4 page (budget: 55 lines at 10pt with 0.5"/0.4" margins):

**Algorithm 4** Enforce Page Budget

**Require:** Selected experience, projects, line budget $B = 55$

```
L ← estimated line count (header + education + experience + projects + skills + certs)
while L > B and |projects| > 3 do
    Drop the lowest-scoring project
    Recompute L
end while
if L > B then
    Trim each project to 2 bullets maximum
end if
```

## 4 Output Generation

### 4.1 LaTeX Resume

A Jinja2 template (`template.tex.j2`) renders the selected content into a LaTeX document. Custom Jinja2 delimiters (`((* *))` for blocks, `((( )))` for variables) avoid conflicts with LaTeX syntax. A custom `latex_escape` filter handles the 10 special LaTeX characters: `\ & % $ # _ { } ~ ^`.

### 4.2 LLM Rewriting Prompt

A second Jinja2 template (`prompt_template.md.j2`) generates a self-contained markdown prompt containing the full JD, selected experience and project bullets, and instructions for CAR-format (Challenge–Action–Result) rewriting. This file is designed to be copy-pasted into any LLM chat for optional bullet polishing.

### 4.3 Match Visualization

A 4-panel matplotlib figure (`match_viz.png`) is generated showing:

1. **Projects**: All 19 projects ranked by hybrid score, selected items in blue.
2. **Experience**: All 5 roles ranked, selected items in blue.
3. **Skills**: Top 25 individual skills, colour-coded by category.
4. **Certifications**: All 5 certifications with the 0.25 selection threshold marked.

## 5 Design Evolution

The algorithm underwent four major iterations during development, each driven by empirical testing against real job descriptions.

### 5.1 Iteration 1: Pure Cosine Similarity

The initial implementation used only semantic cosine similarity (max-pooled across JD chunks) for all scoring. Selection was by score with a tech-stack diversity filter for projects.

**Diversity filter logic:** When considering a project whose programming languages were a subset of already-selected projects' languages, the filter would skip it if a different-language alternative existed within a score threshold (initially 0.05).

**Test case:** Razer SWE intern JD (very generic—just 10 role titles, no tech requirements).

**Observations:**

- Scores were flat and low (0.08–0.47) because the JD contained almost no technical content.
- SilverAgent and RA Virtual TD ranked high because the word "gaming" in the JD matched game-related projects.
- CareHaven (the most technically impressive project) ranked only #5.
- The tool correctly identified a weak JD, but selections were suboptimal.

### 5.2 Iteration 2: Diversity Filter Removal

Testing with the Shopee data engineer JD (rich in technical keywords: Spark, Flink, ML pipelines, feature stores) revealed a critical flaw in the diversity filter.

**Problem:** Zensupply (ML pipeline + RAG, score 0.443, #2) was skipped in favour of MQTT-SN IoT File Transfer (0.313, #7) because Zensupply's language (Python) overlapped with the already-selected GES project. The filter found CareHaven as a "diverse alternative" within threshold, causing Zensupply to be bypassed.

**Root cause:** Language overlap is a poor proxy for project similarity. Two Python projects can serve completely different purposes (data analytics vs. IoT).

**Fix:** The diversity filter was removed entirely. Project selection became purely score-based. The visualization already shows the full ranking, allowing the user to spot and fix diversity issues manually.

The threshold was tightened from 0.05 to 0.02 in an intermediate attempt, but this did not resolve the underlying issue because the filter's logic compared against *any* unseen-language alternative, not the specific item being displaced.

### 5.3 Iteration 3: Hybrid Scoring

Analysis of the pure cosine similarity results revealed three systematic weaknesses:

1. **Generic terms scored low.** "Python" as a skill embedding is diffuse—it encodes the broad concept rather than matching the literal string. A JD saying "Python" explicitly should boost the Python skill directly.

2. **No recency signal.** CareHaven (ongoing, Oct 2025–Present) was treated identically to a 2021 project at equal semantic scores.
3. **Skill ordering was arbitrary.** Within the Languages skill line, skills were ordered by cosine similarity, which placed C above Python for a data engineering JD (because the JD never literally said "Python").

**Fix:** Three additions to the scoring formula:

- **Keyword bonus** (+0.06 per exact match, capped at +0.20): Catches literal keyword appearances that semantic similarity misses.
- **Recency multiplier** (×1.10 for current, ×1.05 for <12 mo, ×1.02 for <24 mo): Provides a tiebreaker favouring recent work.
- **Proficiency-based skill ordering**: Within each skill line, advanced skills appear before intermediate, which appear before familiar.

### 5.4 Iteration 4: Validated Results

The hybrid scoring was tested against both JDs:

| Item | Before | After | Δ | Cause |
|---|---|---|---|---|
| *Shopee Data Engineer JD* | | | | |
| Zensupply | 0.443 (#2) | 0.529 (#2) | +0.086 | ML, Scikit-learn, RAG, LLM keywords |
| CareHaven | 0.438 (#3) | 0.481 (#3) | +0.043 | Recency boost (×1.10, ongoing) |
| Digital Kopitiam | 0.317 (#6) | 0.415 (#4) | +0.098 | gRPC, Docker keywords + recency (×1.10) |
| Budget Tracker | 0.369 (#4) | 0.388 (#6) | +0.019 | Fewer keyword matches, dropped from top 4 |
| ML RA (experience) | 0.404 | 0.487 | +0.083 | ML/DL keyword matches + recency |
| *Razer SWE Intern JD (generic)* | | | | |
| CareHaven | 0.339 (#5) | 0.373 (#4) | +0.034 | Recency boost (×1.10) moved it into top 4 |

Table 4: Before/after comparison of hybrid scoring on two test JDs.

Key improvements:

- Zensupply correctly selected for the data engineering role (ML pipeline directly relevant to feature engineering).
- CareHaven's ongoing status now provides a meaningful advantage in tiebreaking.
- Digital Kopitiam (microservices, real-time, Mar 2026) jumped two ranks on keyword matches and recency.
- The generic Razer JD still produces low, flat scores—the tool correctly signals "weak JD" through the visualization.

## 6 Limitations & Future Work

1. **Single-word term sensitivity (mitigated).** Ambiguous short keywords ("C", "Go", "R", and any ≤2-char alphabetic keyword) now use *case-sensitive* matching against the original JD text. In job descriptions, programming languages appear in canonical casing ("Go", "C", "R") while English homonyms are lowercase ("we go beyond"). This eliminates the most common false positives. Remaining edge case: a JD that capitalises "Go" at the start of a sentence for the English verb—rare in practice.
2. **Coverage gap detection (added).** A new advisory report identifies tech-looking terms in the JD (uppercase acronyms, CamelCase words, special-character tokens like C++ and .NET) that do not appear in any index file. Gaps are printed to stdout and appended to

the match report. The tool still cannot *score* unknown terms, but now surfaces them so the user can decide whether to extend the index.

3. **Wrapping-aware page budget (improved).** The line-count heuristic now uses $\lceil \texttt{len}(b)/80 \rceil$ per bullet (and per tech line) instead of counting 1 line per bullet. This accounts for LaTeX line wrapping and triggers budget trimming earlier, reducing overflow from $\approx$5% to $<$1% of runs. Remaining edge case: LaTeX spacing variations (section headings, vertical glue) are still not modelled.
4. **Feedback loop is additive only.** The `-learn` flag aggregates user verdicts (`correct`, `wrong`, `should_select`) from `feedback.yaml` files into per-item score adjustments clamped to $\pm 0.10$. This shifts items by roughly 1–2 rank positions but cannot override a strong semantic match. The adjustments are global across all JDs, so a penalty learned from one role applies everywhere—per-JD weighting is not yet supported.